ChangeLens: Automated Change Reports for Game Producers from Unity Project Artefacts

COMP3180 RESEARCH PROPOSAL

Name: Raphael Byun

Student ID: 46987282

Topic Category: Developer / production tools

Introduction

Each week a game producer needs to know what changed. A producer plans and tracks the team's work without writing code, so what they know comes from tickets and stand-ups. A two-day refactor can leave no ticket, while a three-line tweak can look like a day's work. Games are harder: most of their files are not code (Pascarella et al., 2018), so a code-only report misses most of the week.

ChangeLens is a Unity Editor tool that reads the files a team already produces and writes a one-page report of what changed, grouped as code, art, audio or scenes. The proposal asks:

> *Which of a producer's information needs can be answered automatically from the artefacts a team already produces — and which can't?*

Two sub-questions answer it. First, which artefact could supply each of ten common producer needs? Eight readers sort them. Second, does naming the changed scenes and prefabs (the Unity asset layer) help a reader understand the week? The same readers compare versions with and without the names. Seven existing approaches each cover one part of this (Appendix A): tickets and commits summarised for engineers, manual entry from game teams, or asset history for artists. None turns the Unity asset layer into a week's account for a producer.

Literature Review

A producer's picture of the week comes from tickets and stand-ups, both written by the people doing the work. Why is that record thinner than the work it describes? This review asks, in turn, where the record fails, what a reader needs and how a game week differs; then why the record thins, where three studies compete (the structure of reporting, the opacity of the tools, or the audience the report is written for); and finally what bar a fix must clear.

Bertram et al. (2010) interviewed and surveyed 15 people across four small, collocated software teams about their issue tracker. Bugs go unfiled because the record fails at the point of entry. One participant said, "often bugs that are that critical don't even get filed because by the time that you would go to file it, the bug is already resolved" (P05, p. 296). Their remedy, a simplified filing interface (p. 297), still asks the under-reporting party to report, so I score reports against the team's history, not its tickets.

Ko et al. (2007), two of them Microsoft Research staff, watched 17 developers at Microsoft and surveyed 42 more. One company can skew how often a need arises but not what kinds exist, so I take the taxonomy, not the base rates. Their 21 information types are each a question, recorded apart from where developers looked, so I classify producers' needs by question first, then ask which artefact answers each. "What have my coworkers been doing?" (§5.7) is a producer's question too; the debugging kinds are not.

Pascarella et al. (2018) analysed the repositories of 30 open-source game projects and 30 non-game systems, comparing which files each changed. Non-game projects have, on average, 80% of files in the Development category, game projects only 30%. In games, sub-teams own non-code categories such as multimedia, so a code-only report is blind to whole disciplines. No Unity or Unreal project appears in its sample, so I measured six Unity repositories myself, including my own team's (Figure 1); the report categorises by file type.

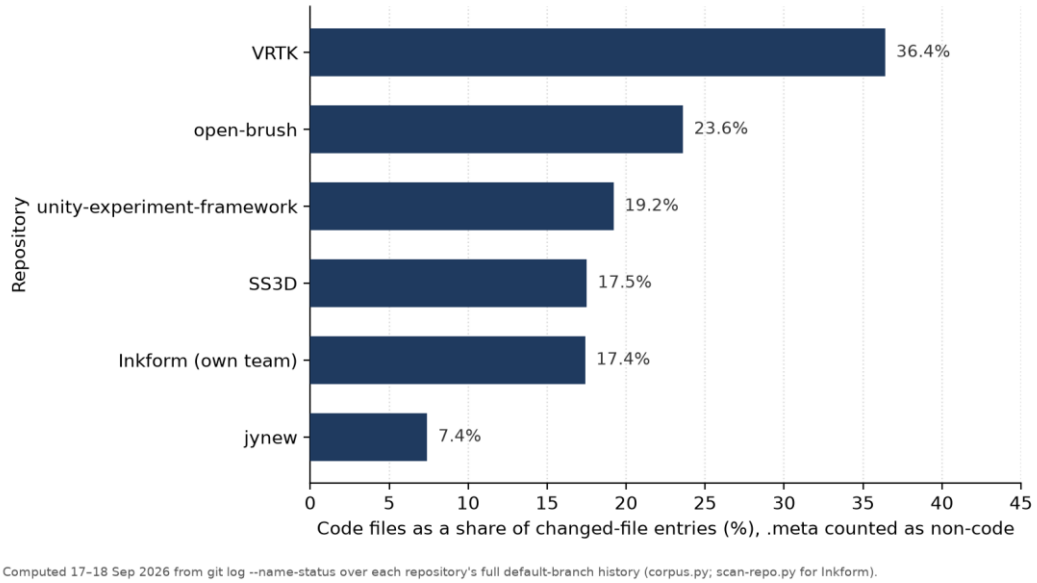


Figure 1. Code share of changed-file entries in six Unity repositories, five open-source and my own team's, .meta files counted as non-code: 7–36%; .meta excluded, 12–57% (Appendix A).

Dourish and Bellotti (1992) watched four groups of three designers solve a 90-minute design problem in a shared editor. Their answer is structural: reporting costs the reporter and pays the group, so it under-delivers by design, and recipients get only what the sender judges relevant, when the sender chooses. Their own example is the version-control edit log, today's commit message; in my corpus, "Fix restart bug (#588)" names an intention while touching 35 files across five categories. Against that they set feedback collected passively as a by-product of the work itself. That is the tool I propose: it reads artefacts instead of asking people. Their evidence is narrow, four groups in one 1992 session, but the mechanism is incentives, not technology.

Whitson (2018) describes 10 weeks of fieldwork with 10 intern developers at a large international game studio in Canada. Developers called the software "voodoo" when their input produced output whose cause they could not predict. One shark model imported into Unity would not animate. Whitson reads this as opacity: the tool hides causes, and blaming the tool rather than a teammate keeps the team moving. Ten interns at one studio make a narrow, interpretive base, so I treat opacity as a hypothesis: extraction names what changed but not why, and the think-aloud follow-up asks what the page leaves unexplained.

Stray et al. (2016) built a grounded theory of the daily stand-up from 12 non-game teams. The meeting degrades when its audience changes: addressed to the manager rather than the team, it becomes status reporting, and nine interviewees said others tuned out (§4.3). That is the third explanation, audience: the record thins because of who it is written for. I take it as a constraint: the report is addressed to the producer, so this mechanism is accepted, not removed, which is why the tool must cost the team nothing. Washburn et al. (2016) sorted 155 developer-written game postmortems into 22 categories, none of them communication; retrospectives record what a team will admit in public, so the absence says little, and I look in artefacts instead.

Maddila et al. (2023) built Nudge, which flags a pull request when it goes quiet too long, and trialled it on 147 Microsoft repositories, their own company's, so, as with Ko et al., I take its design lesson, not its effect size. Nudge predicts when a pull request should have finished, then stays silent while comments, commits or state changes continue within the last 24 hours. Before that check, 86 of 119 dismissed alerts had fired on pull requests active in the previous day. Its limits set the bar for my challenge tier, flagging work gone quiet:

every signal comes from inside one pull request and no verdict is composed across items, so mine must compose across items and read asset activity, or it adds nothing.

Dourish and Bellotti predict that passive extraction repairs the record; Whitson predicts extraction says what changed but not why. The project bets on the structural mechanism. None of these studies observed a game producer; I take their mechanisms and test the transfer. If readers can say what changed but not why, opacity wins. My gap is judging what a team's artefacts can and cannot tell a producer.

Project Proposal

Learning Goals

These capabilities span understand to create (Anderson & Krathwohl, 2001); after this unit I want to be able to:

Understand: explain why a team's production record under-reports its work, distinguishing the structural, opacity and audience explanations. *Measure:* a diary entry stating each mechanism unaided, with its source and a breakdown it predicts.

Apply and Analyse: inspect a production tool against usability heuristics, then test those findings against what its users say, since one evaluator finds only a minority of a tool's problems (Nielsen & Molich, 1990). *Measure:* four tools coded; one reconciliation naming three places where inspection and interviews disagree.

Evaluate: judge whether a team's artefacts can serve a producer's information need, or whether a person must. *Measure:* every need classified against a named artefact, with three needs where I name the artefact I rejected and why.

Create: design a change report that a reader who never opens the codebase can act on, and build it as a Unity Editor tool over the serialised asset layer. *Measure:* a diary entry, written against an unfamiliar Unity week, tying each layout decision to a named reader need.

I was the producer on my COMP3151 team and aim for producer roles, where writing status for non-technical stakeholders is daily work. Six of the nine employers in a ten-advertisement sample from September 2026 ask for progress reporting to leadership, publishers or partners (Appendix A).

Scoping Evidence

Prototype and corpus. The study's material is SS3D (Space Station 3D), an open-source Unity game with 710 commits. ChangeLens is a Unity Editor package (Unity 6.4): it reads a git log export for one week and classifies each changed path by extension and sibling .meta file (Figure 2; Appendix C). The prototype classifies files only. The committed tier, the version I commit to delivering, adds object naming: it reads names from the prefab text and resolves component identifiers through Unity's AssetDatabase (Figure 3). Six of nine sampled SS3D weeks clear the 60% naming target (Deliverable 2; rates in Appendix A), so the two study weeks and a reserve come from those; where no name resolves, the report shows the file path.

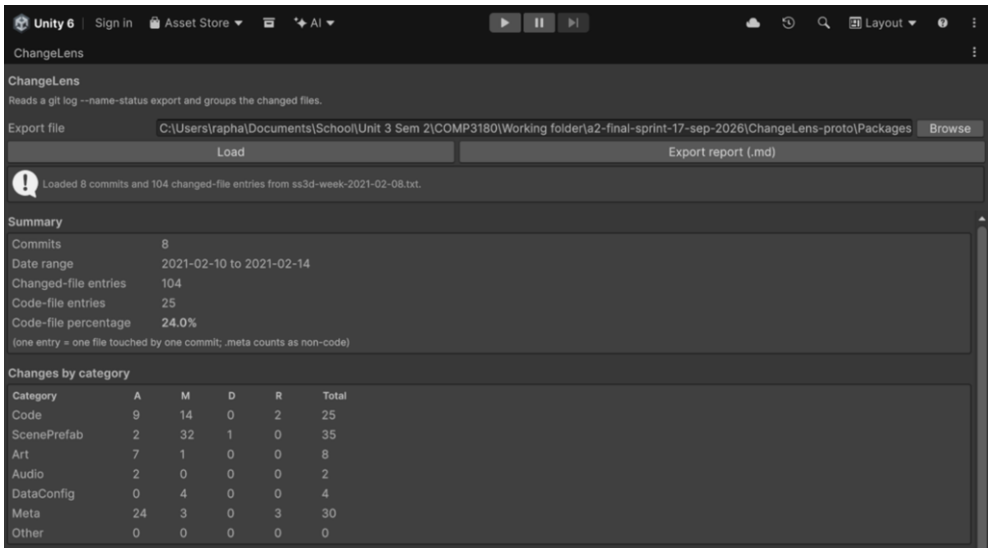


Figure 2. ChangeLens docked in the Unity 6.4 Editor on the SS3D week of 8–14 February 2021 (commits on 10–14 February).

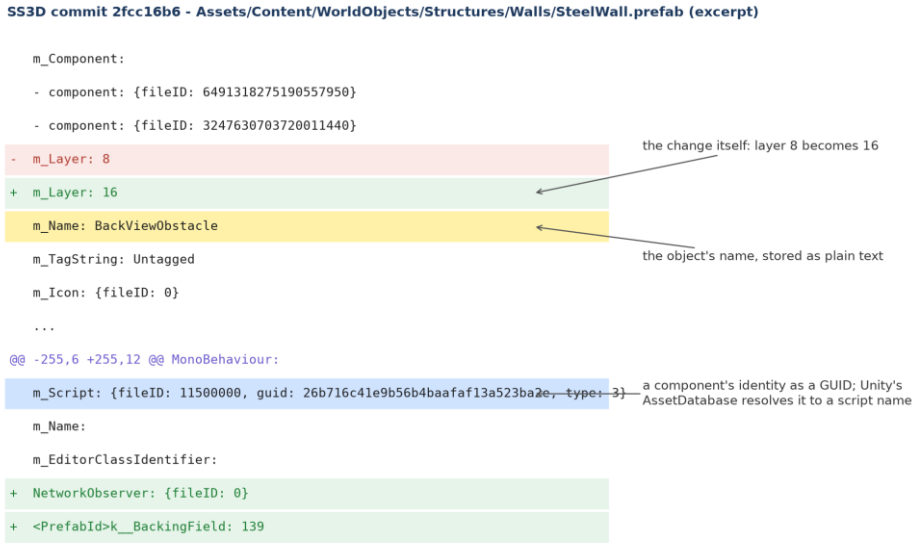


Figure 3. A versioned prefab hunk: object names are plain text; component identifiers are GUIDs the AssetDatabase resolves to names.

No public Unity repository I screened links issues densely enough, nor does my own team's, where none of 126 commits cites a Jira issue (Appendix A). The study therefore reads the asset layer only; the Unity Experiment Framework, the best-linked repository, supplies an illustrative tracker-plus-repository case for Sub-question 1.

Study design. The study is a within-subjects comparison. Eight readers (COMP3151 producers, or teammates briefed into the role) each read two ChangeLens reports for different SS3D weeks: one names the changed scenes and prefabs (treatment), one does not (baseline). Week and order are counterbalanced, and six readers then think aloud while drafting a two-minute stand-up update from the report alone.

Sub-question 1: for each need the tool study surfaces (Deliverable 1), which artefact could supply it, fully or partly: the tracker, the repository or the Unity asset layer? I mark each need derivable, partly derivable or not. All eight readers then classify the same ten needs (the ten Deliverable 1 raises most often) on a one-page sheet; I report percentage agreement and, per reader, Cohen's kappa against my marking; over ten

items these are indicative, not inferential. Each disagreement is a finding about what the artefacts cannot say. Sub-question 2 tests one artefact against one need: does naming the changed scenes and prefabs improve a reader's comprehension of the week?

A pilot runs first, excluded from the data. Each session runs 45 minutes: 5 min of briefing and consent, 12 min per report for two reports (six items each, Appendix B), 6 min for the ten-need sheet, and 10 min of think-aloud for six readers. Week difficulty is a rival explanation, so per-week baseline scores are reported.

I recruit at the Weeks 9–10 COMP3151 and COMP3180 pracs; proxies get a one-page scenario and stand in for studio producers on a week none has worked on, which strips a real producer's context and favours the treatment, so I report the difference as an upper bound. The floor is five of each. Slots are booked by Week 10. Participation is voluntary and reporting de-identified.

The quantitative arm scores the six items against fixed answers at Endsley's (1995) three levels of situation awareness: what changed, what it means, what happens next. One item per report is unanswerable, to catch readers who over-claim from the page. I report each participant's treatment-minus-baseline difference over five scored items; eight readers show direction and failure points, not significance.

The qualitative arm asks what the page leaves a reader needing; in the follow-up the six think-aloud readers explain three classifications. Comments are coded against Ko et al.'s (2007) 21 information types; a quarter is re-coded after a week and agreement is reported.

Before any session, a fidelity check scores every claim in both weeks' reports against each commit's diff, read by hand. It passes when none is contradicted and 90% are supported.

Deliverables

The 65 hours (50 outside class, 15 in prac) cover Weeks 9–13 plus the break.

#	Deliverable	Success measure	Hours	In-prac
1	Production-tool UX study and design spec (Apply and Analyse; Evaluate)	Four free-tier tools coded against Nielsen and Molich (1990) and Nielsen (1994) (4 h); 10 job ads and five G2 reviews coded (1 h); two 20-minute interviews outside the reader pool (2 h); 25 needs classified by artefact source, seeded from Ko et al.'s 21 types and checked against the Unity Experiment Framework (3 h)	10	2
2	ChangeLens Unity Editor package, both conditions (Create)	Generated unedited for every sampled week; 90% of changed files categorised; 60% of changed prefabs and scenes named across the two study weeks	16	5
3	Evaluation protocol and instruments (Evaluate)	Six matched items per week, answerable from the report alone, plus the ten-need sheet; both piloted in Week 11	4	1
4	Fidelity check (Evaluate)	Two team-weeks scored before any session; one hour reserved for repair	3	1
5	Sessions run and coded (Analyse; Evaluate)	Nine 45-minute sessions including the pilot (7 h); recruiting and briefing (4 h); six think-alouds coded from timestamped notes (7 h)	18	4

#	Deliverable	Success measure	Hours	In-prac
6	Evaluation report, instruments, data (Understand; Evaluate)	All six items and both coding arms reported with agreement; claims traceable to appendices	6	1
7	Portfolio video	Three to four minutes showing both conditions; two outside viewers can state its purpose and reader	4	1
8	Development diary	A dated entry each week naming a roadblock or breakthrough	2	0
9	Reflection report (all four goals)	All four goals evidenced; hour variances explained	2	0
	Total		**65**	**15**

The committed tier (11 hours) turns the working extractor (Figure 2) into the producer page, builds both report versions and adds object naming (Figure 3). The stretch tier (3 hours) parses Unity's build report (Appendix C) for failed steps and the three largest asset-size changes. The challenge tier (2 hours) flags work gone quiet on two weeks, gated because Maddila et al. (2023) show ungated alerting misfires. An overrun drops the challenge tier first, then the stretch tier.

A live tracker connection is out of scope; so is a language-model prose layer, which could not pass the fidelity check. The biggest risk is recruitment: readers come from COMP3151 teams building toward their own 3 November release; below five, think-alouds run first and the quantitative arm is descriptive. The second is hand-authoring ground truth: commit messages name the ticket, not the work, so an unscorable week is replaced.

Deliverable 1 runs across the break and Week 9; the committed tier, instruments and fidelity check land in Week 10; the tier freeze, pilot and first sessions in Week 11; the remaining sessions in Week 12; analysis and video in Week 13.

References

Anchorpoint. (n.d.). *Anchorpoint: Version control software for 3D*. Retrieved September 18, 2026, from https://www.anchorpoint.app/

Anderson, L. W., & Krathwohl, D. R. (Eds.). (2001). *A taxonomy for learning, teaching, and assessing: A revision of Bloom's taxonomy of educational objectives*. Longman.

Bertram, D., Voida, A., Greenberg, S., & Walker, R. (2010). Communication, collaboration, and bugs: The social nature of issue tracking in small, collocated teams. In *Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work* (pp. 291–300). ACM. https://doi.org/10.1145/1718918.1718972

Dourish, P., & Bellotti, V. (1992). Awareness and coordination in shared workspaces. In *Proceedings of the 1992 ACM Conference on Computer-Supported Cooperative Work* (CSCW '92) (pp. 107–114). ACM. https://doi.org/10.1145/143457.143468

Endsley, M. R. (1995). Toward a theory of situation awareness in dynamic systems. *Human Factors, 37*(1), 32–64. https://doi.org/10.1518/001872095779049543

HacknPlan. (n.d.). *HacknPlan: The project management tool for game developers*. Retrieved September 17, 2026, from https://hacknplan.com/

Ko, A. J., DeLine, R., & Venolia, G. (2007). Information needs in collocated software development teams. In *Proceedings of the 29th International Conference on Software Engineering* (ICSE '07) (pp. 344–353). IEEE. https://doi.org/10.1109/ICSE.2007.45

Linear. (2026, June 18). *Agent assisted project updates* [Changelog]. https://linear.app/changelog/2026-06-18-agent-assisted-project-updates

Maddila, C., Upadrasta, S. S., Bansal, C., Nagappan, N., Gousios, G., & van Deursen, A. (2023). Nudge: Accelerating overdue pull requests toward completion. *ACM Transactions on Software Engineering and Methodology, 32*(2), Article 35. https://doi.org/10.1145/3544791

McShaffry, M. (2013, March). *Generating useful, meaningful (and beautiful) project data from Perforce* [Conference presentation]. Game Developers Conference 2013, San Francisco, CA, United States. https://www.gdcvault.com/play/1017697/Generating-Useful-Meaningful-(and-Beautiful)

Nielsen, J. (1994). Enhancing the explanatory power of usability heuristics. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (CHI '94) (pp. 152–158). ACM. https://doi.org/10.1145/191666.191729

Nielsen, J., & Molich, R. (1990). Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (CHI '90) (pp. 249–256). ACM. https://doi.org/10.1145/97243.97281

Pascarella, L., Palomba, F., Di Penta, M., & Bacchelli, A. (2018). How is video game development different from software development in open source? In *Proceedings of the 15th International Conference on Mining Software Repositories* (pp. 392–402). ACM. https://doi.org/10.1145/3196398.3196418

Perforce Software. (n.d.). *AI-driven workflows with P4 MCP*. Retrieved September 17, 2026, from https://www.perforce.com/products/p4/p4-mcp

Stray, V., Sjøberg, D. I. K., & Dybå, T. (2016). The daily stand-up meeting: A grounded theory study. *Journal of Systems and Software, 114*, 101–124. https://doi.org/10.1016/j.jss.2016.01.004

Unity Technologies. (n.d.). *Unity Production Pipeline*. Retrieved September 18, 2026, from https://unity.com/features/unity-production-pipeline

Washburn, M., Jr., Sathiyanarayanan, P., Nagappan, M., Zimmermann, T., & Bird, C. (2016). "What went right and what went wrong": An analysis of 155 postmortems from game development. In *Proceedings of the 38th International Conference on Software Engineering Companion* (ICSE '16 Companion) (pp. 280–289). ACM. https://doi.org/10.1145/2889160.2889253

Whitson, J. R. (2018). Voodoo software and boundary objects in game development: How developers collaborate and conflict with game engines and art tools. *New Media & Society, 20*(7), 2315–2332. https://doi.org/10.1177/1461444817715020

ZeroStandup. (n.d.). *ZeroStandup: AI-powered automated daily standups*. Retrieved September 17, 2026, from https://zerostandup.com/

Statement on Generative AI Use

I used generative AI (Claude, via Claude Code) from July to September 2026 to help draft and proofread this proposal, to find and check sources, to mock-mark drafts, and to write the prototype code, the corpus scripts and the figures from my specification. I have read the whole proposal and the sources it cites, made the design decisions, and checked every claim and citation myself; five AI-introduced errors were caught and corrected in that checking. Appendix D lists the session records and verification files, reproduces a transcript excerpt and names the errors caught; the files are available on request.

Appendix A — Scoping data

Vendor sweep (3 August 2026, rechecked 17–18 September 2026). Public product documentation only; studio-internal tooling is not observable.

Tool or source	Reads	Produces	For whom	Does not do
ZeroStandup (n.d.)	Git hosting and issue trackers	Stand-up-style AI summaries	Engineers, team leads, managers	No engine or asset signals; daily altitude only
Linear (2026)	Linear issues and projects, linked Slack channel	Agent-drafted project updates	Product teams	No repository, engine or asset signals
HacknPlan (n.d.)	Manual entry per discipline	Kanban boards, burndown	Game teams	Nothing derived from the repository or engine
Perforce P4 MCP (Perforce Software, n.d.)	Helix Core history, including binary assets	Natural-language answers about history	Developers	No tracker fusion; no producer-facing report
Unity Production Pipeline (Unity Technologies, n.d.)	Builds, assets, version control (closed beta; public availability November 2026)	Web review of renders, build automation	Producers, artists, QA	No change report from tracker plus repository plus asset history; if a report ships before A3, ChangeLens is compared against it

Tool or source	Reads	Produces	For whom	Does not do
Anchorpoint (n.d.)	Git history of Unity, Unreal and Godot projects	Thumbnails, visual review, file locks	Artists, developers	No change report for producers
McShaffry (2013), GDC talk	Perforce logs, mined by hand	Charts for scheduling	Producers	Manual; never productised

Corpus screen (17 September 2026, corpus.py). Thresholds: at least one megabyte of versioned non-code assets, and at least 30 genuine issue references per 100 commits, where a genuine reference cites an issue other than the squash-merged pull request's own number. Five public repositories screened (my own team's is reported below); all exceed the asset threshold; genuine linkage peaked at 20 per 100 commits (Unity Experiment Framework), with SS3D at 9 and open-brush, VRTK and jynew at 0. Code share of changed-file entries, .meta counted as non-code (the Figure 1 series): SS3D 17.5%, open-brush 23.6%, VRTK 36.4%, jynew 7.4%, Unity Experiment Framework 19.2%; with .meta excluded from both numerator and denominator: 33%, 40%, 57%, 12% and 23%. Figure 2's 24% is one week of SS3D on the first basis. The SS3D clone (branch develop, 17 September 2026) has 710 commits from 68 authors.

Naming presence in SS3D weeks (export-week.py, 18–20 September 2026). Share of changed prefab and scene files whose changed text carries an object name: 8–14 Feb 2021, 23 of 26 (88%, 8 commits, the Figure 2 week); 19–25 Jul 2021, 22 of 24 (92%, 7 commits); 14–20 Dec 2020, 30 of 33 (91%, 3 commits); 28 Mar–3 Apr 2022, 28 of 28 (100%, 1 commit); 27 Sep–3 Oct 2021, 22 of 22 (100%, 2 commits); 27 Apr–3 May 2020, 2 of 3 (67%, 5 commits); 3–9 Apr 2023, 10 of 17 (59%, 4 commits); 13–19 Dec 2021, 6 of 13 (46%, 9 commits); 2–8 Feb 2026, 8 of 50 (16%, 3 commits, a bulk NetworkTransform edit). Six of nine clear 60%; the study weeks and the reserve are chosen from the three weeks above 85% with at least three commits (8–14 February 2021, 19–25 July 2021, 14–20 December 2020). The 60% target was set after the first four weeks were measured. My own team's repository (Inkform, COMP3150 semester one, 126 commits, 2,366 entries) measures 17.4% on the first basis and 27.0% on the second; none of its 126 commit messages cites a Jira key, and of the team's 216 Jira issues, 41 mention a commit, branch or pull request in free text but none is linked to one; its sprint pages were written up after the fact for a semester.

Job-advertisement sample (September 2026). Ten producer-family advertisements from LinkedIn, gamejobs.co and workwithindies.com (Seek blocked automated retrieval), nine distinct employers once two identical Aristocrat requisitions are counted once; four Australian; three listings already closed when sampled. Six of the nine employers ask for progress or status reporting to leadership, publishers or partners, for example Light & Wonder ("status reports for leadership") and Devolver Digital ("communication with partners on progress, risks, and changes"). A convenience sample; Deliverable 1's coded scan replaces it.

Appendix B — Draft comprehension items for the 8–14 February 2021 SS3D week

One set per study week (Deliverable 3), scored against fixed answers; L1–L3 are Endsley's levels. Both report versions list the commits and the category counts; the treatment version also names the changed scenes and prefabs.

1. (L1) How many commits landed this week, and which category had the most changed-file entries? *Fixed answer:* eight commits; scenes and prefabs, 35 entries.
2. (L1) Which commit touched the most files? *Fixed answer:* "Fix restart bug (#588)", 35 files.
3. (L2) Was this a code-heavy or a content-heavy week? Give one number from the report that shows it. *Fixed answer:* content-heavy; code is 25 of 104 entries (24%), while scenes, prefabs, art and audio make up 45.
4. (L2) Which commit most likely changed what a player sees or hears, rather than how the game computes? *Fixed answer:* the emergency-light commit (three prefabs, three art files) or the floor-sign commit (four art files, two prefabs); the wires-and-pipes commit (eight code files) is the distractor.
5. (L3) Two commits on 14 February touched airlocks. What would you test first in the next build, and why? *Fixed answer:* airlock interaction and sound, because 18 airlock prefab entries changed in two commits on one day; the treatment version names the nine prefabs.
6. (Unanswerable) Which of these changes were asked for in the tracker? *Fixed answer:* "the report doesn't tell me"; the report reads the repository only.

Appendix C — The ChangeLens report for the sampled week and the build-report probe

Generated by the prototype from the export behind Figure 2 (ss3d-week-2021-02-08-changelens-report.md, 18 September 2026). This is the prototype's full classification; the committed tier's producer-facing page groups Meta and DataConfig into supporting counts beneath code, art, audio and scenes.

Measure	Value
Commits	8 (2021-02-10 to 2021-02-14)
Changed-file entries	104
Code-file entries	25 (24.0%)

Category	Added	Modified	Deleted	Renamed	Total
Code	9	14	0	2	25
ScenePrefab	2	32	1	0	35
Art	7	1	0	0	8
Audio	2	0	0	0	2
DataConfig	0	4	0	0	4
Meta	24	3	0	3	30

Commits, newest first: ea23f08 Add sound fall off to airlocks (#611), 9; 61a96bf Add emergency light and light interactions (#610), 13; a7aab33 Make floor sign openable/closeable (#609), 14; f68b6a3 Add RigidBody to airlocks to fix their interaction (#608), 9; 3855af4 Optional connections for wires & pipes (#607), 10; a326ba1 Rework the container UI (#605), 4; 37e22a3 Fix restart bug (#588), 35 (13 code, 8 scenes and

prefabs, 4 data, 1 art, 9 .meta); 0059f5d Create PepperSpray class to handle interaction with the pepper spray (#604), 10.

Build-report probe (stretch tier). On 18 September 2026 the prototype built a one-object scene for Windows in batch mode and read Unity's BuildReport object: result Succeeded, 25 s, 86.8 MB, 0 errors, 28 timed steps (Postprocess built player 20.0 s), largest packed asset the built-in splash-screen texture at 2,730.7 KB (prototype-logs/buildreport.txt, BatchTools.cs). Failed steps and packed-asset sizes are therefore readable from the same object the stretch tier would parse.

Appendix D — Generative AI use: evidence

Tool: Claude (Anthropic) through Claude Code, July to September 2026. The files named below sit in my COMP3180 working folder and are available on request.

Date (2026)	Activity	Evidence file
28 Jul	Topic exploration and ranked topic options	Topic Council Verdict; Research Brief; Unit-Native Topic Options
3 Aug	Literature research base (one agent per source) and competitor landscape scan	production-reporting-literature/; Competitive Landscape — Production Reporting Tools
8–10 Aug	Second literature base on production-tool UX	production-tool-ux-literature/
28–30 Aug	Source verification, corpus screening, research dossier	a2-recovered-29-aug-2026/
2 Sep	Draft v1, rubric audit, adversarial mock mark, per-source SAFE and DO-NOT-WRITE lists	a2-draft-2-sep-2026/ (draft, audit/, verify/)
17 Sep	Prototype C# written from my specification and tested in Unity 6000.4.10f1; corpus script; figures; audit of Draft v1 by fourteen agents	a2-final-sprint-17-sep-2026/ (ChangeLens-proto/, corpus/, figures/, audit/FINDINGS-v1.md)
17–20 Sep	Seventeen mark-fix-remark rounds against the published rubric, proofreading, fact-checking against the downloaded papers, SS3D week measurements	audit/FINDINGS-v2.md, FINDINGS-v3.md, FINDINGS-round-1.md to -14.md; verified-additions-round-*.md; apply-*.py

Transcript excerpt (Claude Code, 19–20 September 2026). My instruction opening the last loop: "Autonomous improvement loop for my COMP3180 A2 research proposal, continuing from round 11. Run rounds without asking me until a stop condition fires, then report. … Locked facts (never change without asking me): the research question verbatim; the six counted sources (Bertram 2010, Ko 2007, Pascarella 2018, Dourish & Bellotti 1992, Whitson 2018, Maddila 2023) … hours summing to 65 with 15 in-prac … Every number, quotation and attribution must be covered by a SAFE list." Later that night: "for this loop do not ask me questions as I will be going to sleep, just choose by yourself and keep on improving the document." The

tool's reply after one round: "Round 11 is complete: Word body 2,470 (at the ceiling), 11 pages, markdown, .docx and PDF in sync, findings and SAFE list written."

Errors the AI introduced and verification caught. (1) A fabricated quotation ("producers as risk-communication ambassadors") that exists in no source, removed 3 August. (2) Whitson's 2020 *Games and Culture* article substituted for the 2018 *New Media & Society* paper, reverted 21–22 August. (3) Two paper counts (96 and 166) conflated between a 2021 testing survey and a 2022 workshop dataset, corrected 3 August. (4) An "articulation failure" mechanism attributed to Whitson (2018) that the paper does not support, corrected 2 September to opacity plus blame on the tool. (5) Nudge notifications described as reaching "the blocked developer", which no source states, corrected 17 September.

What I did myself. Chose the topic and the research question; held the staff 1-on-1s that set the direction; ran the two producer interviews; made every scope and design decision; reviewed and edited each section; ran the tool; read the proposal and the sources it cites in full.